

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	iv
Índice de figuras	v
Introducción	1
1 Propuestas analizadas	4
1.1 XV6	5
1.1.1 Nombre del proyecto o sistema operativo	5
1.1.2 Enlace al repositorio y/o documentación oficial	5
1.1.3 Objetivo del proyecto	5
1.1.4 Lenguaje(s) de implementación	6
1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)	6
1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)	8
1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.) . . .	16
1.1.8 Nivel de complejidad y accesibilidad para estudiantes	17
2 Comparación técnica	18

3	Análisis crítico	19
	Referencias	20

Índice de tablas

1.1	Componentes principales del sistema operativo XV6	9
1.2	Herramientas de desarrollo para xv6	17

Índice de figuras

1.1	Archivos fuente del kernel de xv6 y sus funciones principales.	7
1.2	Layout del espacio de direcciones de un proceso xv6 . El proceso ve en su espacio un segmento de texto (solo lectura/ejecución), un segmento de datos/heap (lectura-escritura), y una pila de usuario (lectura-escritura hacia abajo)	12

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 XV6

1.1.1 Nombre del proyecto o sistema operativo

El sistema operativo xv6 es una versión moderna y simplificada del Unix Sexta Edición (V6), creada con fines educativos. Fue desarrollado por el MIT en 2006 para el curso de Engineering Operating Systems (6.828) (MIT PDOS, 2024a), con el objetivo de ofrecer una herramienta didáctica más accesible y actual que el Unix original, el cual estaba escrito en un dialecto antiguo de C (pre-ANSI). Esta reimplementación mantiene la esencia y estructura del Unix clásico, pero utiliza un código más claro y compatible con los entornos de aprendizaje contemporáneos.

1.1.2 Enlace al repositorio y/o documentación oficial

- **Enlace al repositorio oficial:** <https://github.com/mit-pdos/xv6-public>
- **Enlace al libro oficial:** <https://pdos.csail.mit.edu/6.828/2018/xv6/book-rev10.pdf>

1.1.3 Objetivo del proyecto

El propósito principal de xv6 es educativo y experimental. Fue diseñado para funcionar como un kernel de estudio que permita a los estudiantes comprender los principios fundamentales de Unix dentro de un curso semestral (Wikipedia contributors, 2024). A diferencia de sistemas operativos más complejos como Linux o BSD, xv6 mantiene una estructura lo bastante simple como para ser analizada y entendida en su totalidad durante una asignatura, pero al mismo tiempo conserva los componentes esenciales y la lógica interna de un sistema Unix real.

1.1.4 Lenguaje(s) de implementación

El núcleo de xv6 está escrito principalmente en ANSI C, con pequeñas porciones en lenguaje ensamblador (por ejemplo, para rutinas de arranque y manejo de interrupciones)(MIT PDOS, 2024a) (pag.7). La versión original de x86 es C (para x86-32); la versión reciente soporta RISC-V también escrita en C

1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Como explica la documentación, xv6 es un kernel monolítico: toda la funcionalidad del sistema corre con privilegios de kernel, sin servidores externos (MIT PDOS, 2024a) (pag.25) En Xv6, el kernel sigue el diseño clásico de los sistemas Unix. Funciona como un núcleo único que brinda servicios a múltiples procesos, los cuales se ejecutan en un espacio de usuario independiente. Cada proceso va alternando entre dos modos de ejecución:

- **En el modo usuario:** el proceso ejecuta su propio código y realiza operaciones computacionales básicas.
- **En el modo kernel:** es el sistema operativo quien toma el control para ejecutar las llamadas al sistema solicitadas.

Este cambio ocurre cuando un proceso necesita un servicio del sistema operativo: el hardware cambia automáticamente a un modo de mayor privilegio (modo supervisor) y transfiere la ejecución al kernel. Una vez que este completa la tarea, devuelve el control al proceso en el espacio de usuario.(MIT PDOS, 2024a) (pag.9)

El código fuente del kernel de xv6 se organiza en el directorio `kernel/`. Cada

archivo implementa un subsistema: por ejemplo, `proc.c` gestiona procesos y planificación, `file.c` maneja descriptores de archivo, `fs.c` el sistema de archivos, `pipe.c` las tuberías, `trap.c` e `idtrnv`: trampolín.asm, etc. El diseño es monolítico pero modular interno: los archivos reflejan componentes del sistema. La Figura de abajo (Figura 2.2 del manual) muestra los principales archivos y su función.

File	Description
<code>bio.c</code>	Disk block cache for the file system.
<code>console.c</code>	Connect to the user keyboard and screen.
<code>entry.S</code>	Very first boot instructions.
<code>exec.c</code>	<code>exec()</code> system call.
<code>file.c</code>	File descriptor support.
<code>fs.c</code>	File system.
<code>kalloc.c</code>	Physical page allocator.
<code>kernelvec.S</code>	Handle traps from kernel, and timer interrupts.
<code>log.c</code>	File system logging and crash recovery.
<code>main.c</code>	Control initialization of other modules during boot.
<code>pipe.c</code>	Pipes.
<code>plc.c</code>	RISC-V interrupt controller.
<code>printf.c</code>	Formatted output to the console.
<code>proc.c</code>	Processes and scheduling.
<code>sleeplock.c</code>	Locks that yield the CPU.
<code>spinlock.c</code>	Locks that don't yield the CPU.
<code>start.c</code>	Early machine-mode boot code.
<code>string.c</code>	C string and byte-array library.
<code>switch.S</code>	Thread switching.
<code>syscall.c</code>	Dispatch system calls to handling function.
<code>sysfile.c</code>	File-related system calls.
<code>sysproc.c</code>	Process-related system calls.
<code>trampoline.S</code>	Assembly code to switch between user and kernel.
<code>trap.c</code>	C code to handle and return from traps and interrupts.
<code>uart.c</code>	Serial-port console device driver.
<code>virtio_disk.c</code>	Disk device driver.
<code>vm.c</code>	Manage page tables and address spaces.

Figura 1.1: Archivos fuente del kernel de xv6 y sus funciones principales.

Fuente: Obtenido de (MIT PDOS, 2024a) *xv6: a simple, Unix-like teaching operating system*.

1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)

XV6 implementa los componentes esenciales de un sistema operativo Unix clásico, adaptados a un entorno educativo. A continuación, se describen los principales subsistemas y su funcionamiento técnico (ver Tabla ??).

Tabla 1.1: Componentes principales del sistema operativo **XV6**

Componente / Sub-sistema	Archivo(s) principal(es)	Funcionamiento técnico (idea clave)
Gestión de procesos y planificador	<code>proc.c</code> , <code>proc.h</code>	Define la estructura <code>proc</code> y el planificador round-robin por CPU; maneja el ciclo de vida de procesos (<code>fork</code> , <code>exec</code> , <code>exit</code> , <code>wait</code>).
Gestión de memoria virtual	<code>vm.c</code> , <code>kalloc.c</code>	Implementa tablas de páginas por proceso (Sv39) y asignador físico mediante <code>kalloc/kfree</code> .
Sistema de archivos	<code>fs.c</code> , <code>file.c</code> , <code>log.c</code>	FS tipo Unix con <code>inode</code> , directorios, caché de bloques y journaling mediante write-ahead log.
Interfaz de <code>syscalls</code>	<code>syscall.c</code> , <code>sysproc.c</code> , <code>usys.S</code>	Tabla de <code>syscalls</code> y despacho de traps del modo usuario al kernel (<code>fork</code> , <code>read</code> , <code>write</code> , etc.).
Sincronización	<code>spinlock.c</code> , <code>sleeplock.c</code>	Provee <code>spinlock</code> y <code>sleeplock</code> para exclusión mutua y bloqueos dormibles.
Entrada/Salida y drivers	<code>console.c</code> , <code>uart.c</code> , <code>virtio_disk.c</code>	Drivers básicos UART y VirtIO integrados con la caché de bloques.
Programas de usuario y shell	<code>user/sh.c</code> , <code>user/*.c</code>	Shell simple y utilidades de usuario para probar <code>syscalls</code> .

Gestión de procesos y planificación

En xv6, cada proceso representa una única unidad de ejecución con su propio hilo. Cada uno mantiene su contexto independiente (registros, tabla de páginas, etc.), que se guarda y restaura cuando el sistema cambia entre procesos. El kernel almacena toda esta información en una estructura por proceso, similar a otros sistemas Unix. MIT PDOS (2024a)(pag.19)

La creación de procesos sigue el modelo tradicional: `fork()` genera un proceso hijo que es una copia del padre, mientras que `exec()` permite cargar un programa completamente nuevo. Todos los procesos comparten el mismo entorno básico, sin distinciones entre diferentes usuarios. La planificación utiliza un algoritmo round-robin muy básico. Cada CPU tiene su propio planificador que recorre cíclicamente la lista de procesos listos para ejecutar, asignando a cada uno un turno de tiempo fijo. No existen prioridades ni políticas complejas, manteniendo el diseño deliberadamente simple. MIT PDOS (2024a)(pag.82)

Gestión de memoria virtual

Xv6 gestiona la memoria mediante paginación, donde cada proceso opera en su propio espacio de direcciones virtuales independiente. El hardware (específicamente RISC-V) se encarga de traducir estas direcciones virtuales a físicas usando tablas de páginas. Aunque RISC-V soporta direcciones de 64 bits, xv6 utiliza solo los 38 bits inferiores, lo que teóricamente permite a cada proceso acceder hasta aproximadamente 256 GB de memoria. MIT PDOS (2024a)(pag.38)

El espacio de direcciones de cada proceso sigue la convención Unix clásica: La

distribución de la memoria sigue el diseño clásico de Unix:

- **Segmento de Código:** Ubicado en las direcciones más bajas (cerca de 0), con permisos de solo lectura y ejecución para proteger el código del programa.
- **Datos y Heap:** Situados a continuación del código, con permisos de lectura y escritura. El heap puede expandirse dinámicamente cuando el programa solicita más memoria mediante la llamada `sbrk`.
- **Pila del Usuario:** Se encuentra en la parte superior del espacio de direcciones y crece hacia abajo. También tiene permisos de lectura y escritura. Para detectar desbordamientos, existe una “página guardia” justo debajo de la pila; si un programa intenta escribir en esta zona, el kernel intercepta el fallo y termina el proceso.

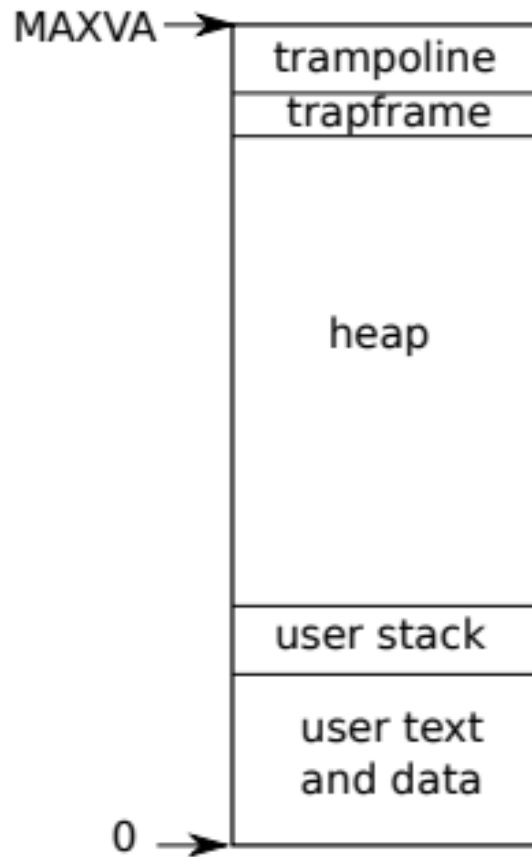


Figura 1.2: Layout del espacio de direcciones de un proceso xv6 . El proceso ve en su espacio un segmento de texto (solo lectura/ejecución), un segmento de datos/heap (lectura-escritura), y una pila de usuario (lectura-escritura hacia abajo)

Fuente: Obtenido de (MIT PDOS, 2024a) *xv6: a simple, Unix-like teaching operating system*.

Sistema de archivos

El sistema de archivos de xv6 organiza la información de forma jerárquica y utiliza journaling para garantizar la integridad de los datos. Su arquitectura sigue un diseño por capas similar a Unix, donde cada nivel se especializa en una función específica:

desde el acceso físico al disco, pasando por la memoria caché de bloques, el registro transaccional, hasta la gestión de archivos, directorios y rutas. (Pekopeko11, 2024).

Organización en Disco El disco se estructura en secciones bien definidas:

- **Bloque 0:** Contiene el código de arranque
- **Bloque 1:** Almacena el “superbloque” con los metadatos del sistema de archivos
- **Bloques siguientes:** Albergan los inodos (que representan archivos)
- **Secciones posteriores:** Incluyen mapas de bits para espacios libres y los bloques de datos reales
- **Zona final:** Reservada para el journal o registro transaccional

Mecanismo de Journaling Para prevenir corrupción de datos, xv6 emplea un sistema transaccional:

- Antes de escribir cambios definitivos en el disco, estos se registran temporalmente en el journal
- Solo cuando la transacción se completa satisfactoriamente, los cambios se aplican permanentemente
- En caso de fallos (como cortes de energía), el sistema puede recuperar el estado coherente revisando el journal

Gestión de Archivos y Directorios Los directorios son archivos especiales que contienen asociaciones entre nombres y inodos. El sistema resuelve las rutas de

manera secuencial (por ejemplo, para `/usr/bin/ls`, busca primero `usr`, luego `bin`, y finalmente `ls`).

Las operaciones básicas (`open`, `read`, `write`, `close`) trabajan mediante descriptores de archivo. Internamente, estas llamadas utilizan la memoria caché de bloques: `read` carga bloques del disco a memoria, mientras `write` los marca como modificados para su posterior escritura transaccional. (MIT PDOS, 2024a)(pag. 85-86)

Interfaz de llamadas (syscalls)

Xv6 implementa una interfaz de llamadas al sistema (syscalls) que permite a los programas de usuario solicitar servicios del kernel. Esta interfaz sigue el modelo tradicional de Unix, donde las llamadas al sistema son invocadas mediante interrupciones de software que cambian el modo de ejecución del CPU del modo usuario al modo kernel.(MIT PDOS, 2024a)(pag.9) Cuando un proceso de usuario realiza una syscall, el hardware genera una interrupción que transfiere el control al kernel. El kernel entonces identifica la syscall solicitada mediante un número único y despacha la ejecución a la función correspondiente en el kernel. Una vez que la syscall se completa, el kernel devuelve el control al proceso de usuario, junto con cualquier valor de retorno necesario.

Sincronización

Xv6 proporciona mecanismos básicos de sincronización para gestionar el acceso concurrente a recursos compartidos. Los principales mecanismos son los `spinlocks` y `sleeplocks`.(MIT PDOS, 2024a)(pag.65)

Entrada/Salida y drivers

Los dispositivos en xv6 se integran en el sistema bajo el principio de que “todo es un archivo”. Esto se logra mediante **archivos de dispositivo**, creados con la llamada al sistema `mknod`.

- `mknod(path, major, minor)`: Crea un archivo especial que representa un dispositivo. Los parámetros `major` (mayor) y `minor` (menor) son números que identifican de forma única al controlador (*driver*) del kernel y al dispositivo específico.

Controladores de Dispositivos (Drivers) El kernel incluye controladores mínimos pero funcionales para hardware esencial. Algunos ejemplos son:

- `uart.c` para el puerto serie.
- `virtio_disk.c` para el acceso al disco.
- `timer.c` para el temporizador del sistema.

Xv6 prioriza la simplicidad, por lo que solo implementa los controladores estrictamente necesarios para el arranque y operación básica, sin soporte para red u otros periféricos complejos.

Programas de usuario y shell

Xv6 incluye un shell básico inspirado en el Bourne shell, programado completamente en C. Este intérprete de comandos realiza las funciones esenciales: leer órdenes del

usuario, soportar operaciones básicas como tuberías y redirecciones, y ejecutar programas creando nuevos procesos. **Funcionamiento del shell** Cuando introduces un comando como `ls | grep txt`, el shell:

- Crea dos procesos hijos independientes
- Conecta la salida del primer proceso (`ls`) con la entrada del segundo (`grep`) mediante una tubería
- Espera a que ambos procesos terminen su ejecución

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Para construir y ejecutar xv6 se emplean compiladores GNU y emuladores comunes. El código se compila con GCC (o clang compatibles) apuntando a binarios ELF para x86/RISC-V (MIT PDOS, 2024b). En sistemas Unix/Linux se puede usar el compilador nativo; en macOS es necesario un cross-compiler para generar ELFp-dos.csail.mit.edu. Normalmente se ejecuta en máquinas virtuales mediante QEMU como emulador de hardware.

Tabla 1.2: Herramientas de desarrollo para xv6

Herramienta	Uso / Descripción
GCC (GNU C Compiler)	Compila el kernel y programas en C para x86-32 o RISC-V (binarios ELF).
QEMU	Emulador de hardware (Intel x86 o RISC-V) para cargar y probar xv6.
Cross-compiler	Necesario en macOS u otros sistemas para generar ejecutables ELF.

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Xv6 fue diseñado específicamente para ser un sistema operativo ligero y educativo. Su código completo consta de aproximadamente 10,000 líneas (equivalente a unas 99 páginas impresas) Su simplicidad relativa (ausencia de capas complejas como módulos dinámicos) lo hace muy accesible para estudiantes de OS. Aun así, cubre los conceptos esenciales de concurrencia, paginación, sistemas de archivos y llamadas al sistema típicos de un Unix real(MIT PDOS, 2024a) (pag.10)

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

MIT PDOS (2024a). xv6: a simple, Unix-like teaching operating system. Accedido: 21 septiembre 2025.

MIT PDOS (2024b). xv6 source and text. Accedido: 21 septiembre 2025.

Pekopeko11 (2024). Xv6 File System and Log Design. accessed: 21-septiembre-2025.

Wikipedia contributors (2024). Xv6. accessed: 21-septiembre-2025.